

Penyelesaian Permainan Queens Linkedln Menggunakan Algoritma Brute Force Murni

Bernhard Aprillio Pramana

13524074

1. Pendahuluan

Queens adalah gim logika yang tersedia pada situs jejaring profesional LinkedIn. Tujuan dari gim ini adalah menempatkan queen pada sebuah papan persegi berwarna sehingga terdapat hanya satu queen pada tiap baris, kolom, dan daerah warna. Selain itu, satu queen tidak dapat ditempatkan bersebelahan dengan queen lainnya, termasuk secara diagonal.

Permasalahan Queens dapat dimodelkan sebagai persoalan pencarian kombinatorial, di mana setiap kemungkinan penempatan queen pada papan harus diperiksa apakah memenuhi seluruh aturan yang diberikan. Dengan bertambahnya ukuran papan, jumlah kemungkinan konfigurasi juga meningkat secara signifikan sehingga diperlukan pendekatan algoritmik yang sistematis untuk menjelajahi ruang solusi yang ada.

Pendekatan brute force dilakukan dengan cara mencari seluruh kemungkinan penempatan queen pada papan dan memeriksa setiap konfigurasi untuk menentukan apakah konfigurasi tersebut valid dan memiliki penyelesaian atau tidak. Untuk mempermudah visualisasi proses pencarian solusi secara *realtime*, program diimplementasikan dalam bahasa Python dengan GUI menggunakan *library* Tkinter.

2. Algoritma yang Digunakan

Algoritma brute force adalah metode penyelesaian masalah dengan cara mencoba seluruh kemungkinan solusi yang mungkin tanpa menggunakan optimasi, heuristik, maupun pemangkasan ruang pencarian. Pendekatan ini menjamin bahwa setiap konfigurasi solusi akan diperiksa satu per satu hingga ditemukan solusi yang memenuhi seluruh aturan, atau hingga seluruh kemungkinan habis dicoba. Pada permasalahan Queens, algoritma brute force digunakan untuk mengeksplorasi seluruh kemungkinan penempatan queen pada papan permainan sesuai dengan aturan yang telah ditentukan.

Papan permainan berbentuk persegi dengan ukuran $N \times N$. Karena aturan permainan mewajibkan tepat satu queen pada setiap baris, maka solusi dimodelkan menggunakan sebuah array satu dimensi queen dengan panjang N .

Indeks array menyatakan baris papan, sedangkan nilai pada indeks tersebut menyatakan kolom tempat queen diletakkan. Papan warna atau region direpresentasikan sebagai matriks dua dimensi berisi karakter yang menunjukkan daerah warna pada setiap sel papan. Dengan pemodelan ini, setiap kemungkinan solusi merupakan satu kombinasi penempatan queen pada seluruh baris papan.

Langkah kerja algoritma brute force dalam program ini adalah sebagai berikut.

- a. Program memulai dengan papan kosong dan array queen yang belum berisi penempatan valid.
- b. Queen dicoba ditempatkan pada seluruh kolom yang tersedia pada baris pertama.

- c. Untuk setiap pilihan pada baris pertama, program melanjutkan ke baris kedua dan kembali mencoba seluruh kemungkinan kolom. Proses ini berlanjut hingga baris terakhir papan terisi queen.
- d. Setelah seluruh baris terisi, terbentuk satu konfigurasi lengkap penempatan queen.
- e. Konfigurasi tersebut kemudian diperiksa apakah tidak ada dua queen pada kolom yang sama, tidak ada dua queen yang bersebelahan, dan tidak ada dua queen yang berada pada daerah warna yang sama.
- f. Jika konfigurasi valid, algoritma menghentikan pencarian dan menampilkan solusi.
- g. Jika konfigurasi tidak valid, algoritma melanjutkan pencarian ke konfigurasi berikutnya.
- h. Proses berakhir ketika solusi ditemukan atau ketika seluruh kemungkinan konfigurasi telah diperiksa.

3. Implementasi dan Source Code Program

Program diimplementasikan dengan bahasa Python dan GUI dengan *library* Tkinter. Modul time untuk menghitung waktu pemrosesan, modul OS untuk pengelolaan penyimpanan input dan output.

Papan permainan direpresentasikan sebagai matriks dua dimensi berukuran $N \times N$ yang dibaca dari file .txt. Setiap sel papan berisi satu karakter alfabet yang merepresentasikan daerah warna. Ukuran papan ditentukan dari jumlah baris pada berkas input. Penempatan queen disimpan dalam sebuah list satu dimensi bernama queen, di mana indeks merepresentasikan baris dan nilai pada indeks tersebut merepresentasikan kolom tempat queen diletakkan. Nilai -1 menunjukkan bahwa pada baris tersebut belum ada queen yang ditempatkan.

```
def loadBoard():
    global board, ukuranBoard, queen, inputFileName, colorMap
    path = filedialog.askopenfilename(filetypes=[("SixSeven", "*.txt")])
    if not path:
        return
    inputFileName = path
    with open(path) as input:
        board = [list(line.strip()) for line in input if line.strip()]
    ukuranBoard = len(board)
    queen = [-1] * ukuranBoard
    canvas.config(width=ukuranBoard*cellSize, height=ukuranBoard*cellSize)
    colors = set(cell for row in board for cell in row)
    if len(colors) > ukuranBoard:
        messagebox.showerror("Invalid Board", f"Papan tidak valid: {len(colors)} region > ukuran papan {ukuranBoard}")
        board = []
        ukuranBoard = 0
        queen = []
        inputFileName = ""
        colorMap = {}
        return
    colorMap = {}
    for alphabet in colors:
        idx = ord(alphabet) - ord('A')
        color = (idx * 67) % 360
        r, g, b = coloring(color)
        colorMap[alphabet] = f"#{r:02x}{g:02x}{b:02x}"
    draw()
```

Algoritma yang digunakan adalah exhaustive search murni. Program mencoba seluruh kemungkinan penempatan queen pada setiap baris tanpa melakukan backtracking. Proses pencarian dilakukan secara rekursif melalui fungsi solveHelper(x), dengan

parameter x menyatakan baris yang sedang diproses. Untuk setiap baris, algoritma mencoba menempatkan queen di seluruh kolom yang mungkin. Setelah seluruh baris terisi, konfigurasi papan diperiksa validitasnya menggunakan fungsi safe().

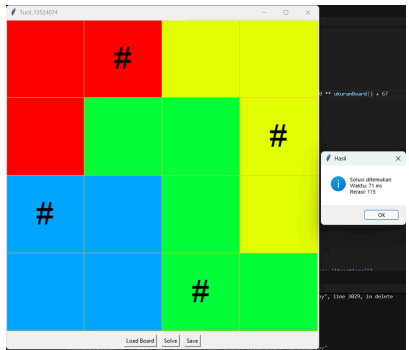
```
def solveHelper(x):
    global iterations
    if x == ukuranBoard:
        iterations += 1
        if ukuranBoard <= 6:
            finalFreq = 10
        if ukuranBoard > 6:
            finalFreq = (10 ** ((ukuranBoard // 2) + 2)) * (1 + iterations // (10 ** ukuranBoard)) + 67
        if iterations % int(finalFreq) == 0:
            draw()
            myKisah.update()
        if safe():
            return True
        return False

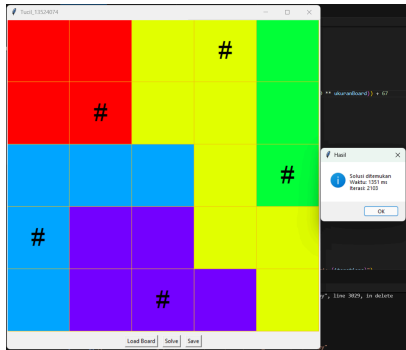
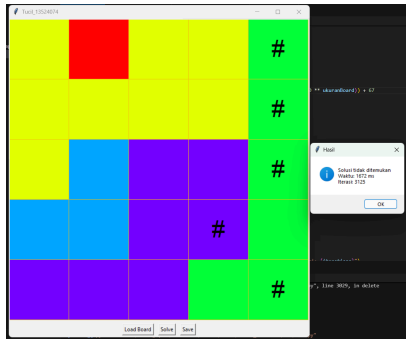
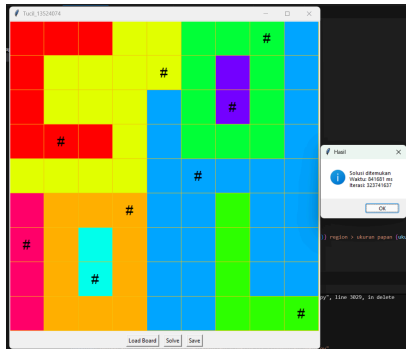
    for i in range(ukuranBoard):
        queen[x] = i
        if solveHelper(x + 1):
            return True
        queen[x] = -1
    return False

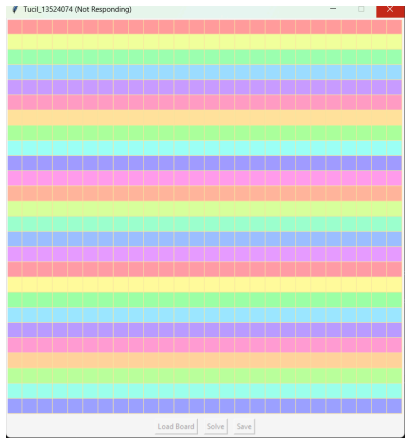
def solve():
    global iterations
    iterations = 0
    start = time.time()
    found = solveHelper(0)
    elapsed = int((time.time() - start) * 1000)
    if found:
        draw()
        status = "ditemukan" if found else "tidak ditemukan"
        messagebox.showinfo("Hasil", f"Solusi {status}\nWaktu: {elapsed} ms\nIterasi: {iterations}")
```

Setiap kali sebuah konfigurasi terbentuk, variabel iterations akan ditambah satu. Program juga mengukur waktu eksekusi pencarian solusi dalam satuan milidetik (ms). Informasi jumlah iterasi dan waktu eksekusi ditampilkan kepada pengguna setelah proses pencarian selesai.

4. Hasil dan Pengujian

No	Ukuran Papan	Input	Output
1	4x4	AABB ACCB DDCB DDCC	 Solusi ditemukan

			Waktu: 71ms Iterasi: 115
2	5x5	AABBC AABBC DDDBC DEEBB DEEEB	 Solusi ditemukan Waktu: 1351ms Iterasi: 2103
3	5x5 Tidak ada penyelesaian	BABBC BBBBC BDEEC DDEEC EEEC	 Solusi tidak ditemukan Waktu: 1672ms Iterasi: 3125
4	8x8	AAABBCCCD ABBBBCECD ABBBDCED AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH	 Solusi tidak ditemukan Waktu: 841681ms Iterasi: 323741637

5	26x26	AAAAAAAAAAAAAA AAAAAAAAAAAAAA AA BBBBBBBBBBBBBB BBBBBBBBBBBBBB CCCCCCCCCCCCCC CCCCCCCCCCCCCC DDDDDDDDDDDDD DDDDDDDDDDDDD DD EEEEEEEEEEEEEEE EEEEEEEEEEE FFFFFFFFFFFFFFFF FFFFFFFFFF GGGGGGGGGGGG GGGGGGGGGGGG GG HHHHHHHHHHHH HHHHHHHHHHHH HH IIIIIIIIIIIIIIIII JJJJJJJJJJJJJJJJJ JJJ KKKKKKKKKKKKK KKKKKKKKKKKKK KK LLLLLLLLLLLLLLL LLLLLLLLLLL MMMMMMMMMMMM MMMMMMMMMMMM MMMMMM NNNNNNNNNNNNN NNNNNNNNNNNNN NN OOOOOOOOOOOO OOOOOOOOOOOO OO PPPPPPPPPPPPPPP PPPPPPPPP QQQQQQQQQQQQ QQQQQQQQQQQQ QQ RRRRRRRRRRRRR RRRRRRRRRRRRR SSSSSSSSSSSSSS SSSSSSSSS	 Papan terlalu besar sehingga program tidak kuat untuk menjalankan dan mencari solusi
---	-------	--	---

		TTTTTTTTTTTTTTTT TTTTTTTTTTTT UUUUUUUUUUUUUU UUUUUUUUUUUUUU UU VVVVVVVVVVVVVV VVVVVVVVVVVVVV VV WWWWWWWWWW WWWWWWWWWW WWWWWWWWWW XXXXXXXXXXXXXX XXXXXXXXXXXXXX XX YYYYYYYYYYYYYY YYYYYYYYYYYYYY YY ZZZZZZZZZZZZZZZZ ZZZZZZZZZZZZ	
--	--	--	--

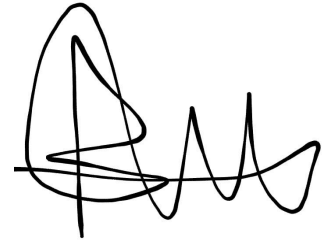
5. Lampiran

http://github.com/IzzyMeow/Tucil1_13524074

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	V	
2	Program berhasil dijalankan	V	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	V	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	V	
5	Program memiliki Graphical User Interface (GUI)	V	
6	Program dapat menyimpan solusi		V

	dalam bentuk file gambar		
--	--------------------------	--	--

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, consisting of a large, stylized initial 'B' followed by a series of connected loops and a final horizontal stroke.

Bernhard Aprillio Pramana